



Robótica: Relatório sobre o método da Janela Dinâmica

Discente: Adhemar Luiz Cavalcanti Silva Rocha Neto

Pontos levantados em Fox, Burgard e Thrun

Ao analisar o arquivo e o código base, percebe-se que é a união entre a base matemática e os critérios de segurança para tornar o código base funcional

1. Operação Direta no Espaço de Velocidades: seguindo a lógica do arquivo, o planeamento não deve ser feito no espaço cartesiano (x,y), mas sim diretamente no espaço das velocidades. No código base fica representado em:

```
157     # evaluate all trajectory with sampled input in dynamic window
158     for v in np.arange(dw[0], dw[1], config.v_resolution):
159         for y in np.arange(dw[2], dw[3], config.yaw_rate_resolution):
```

2. O Conceito de Trajetórias de Curvatura (Arcos): no arquivo, Fox aplica os intervalos de tempo curtos, a trajetória do robô pode ser aproximada por arcos de circunferência determinados pelo par(velocidade linear e velocidade angular). No artigo, essa prática simplifica o tratamento de colisão , pois o robô segue uma linha curva previsível.No código base, a função **predict_trajectory** utiliza exatamente este modelo cinemático para projetar onde o robô estará nos próximos 3 segundos.
3. Velocidades Admissíveis (Va) e Distância de Travagem: Este é o ponto mais crítico para a segurança. Fox define que uma velocidade só é admissível se o robô conseguir parar antes de colidir com o obstáculo mais próximo. Esta lógica está simplificada na função **calc_obstacle_cost**. Se a distância até o obstáculo for menor que o raio do robô em qualquer ponto da trajetória prevista, a velocidade é descartada com custo infinito:

```
210         if np.array(r <= config.robot_radius).any():
211             return float("Inf")
```

4. A Função de Maximização (Objetivo): Fox propõe uma função que deve ser maximizada para encontrar o movimento "ótimo". Ela é composta por:
 - a. **Target Heading:** Alinhamento com o objetivo;
 - b. **Clearance:** Distância dos obstáculos;
 - c. **Velocity:** Incentivo à rapidez.

No código, isso é implementado como uma **Minimização de Custos** (que matematicamente é o inverso da maximização de Fox):

```
167         final_cost = to_goal_cost + speed_cost + ob_cost
```

5. Janela Dinâmica Baseada na Aceleração: Fox introduziu o termo "Janela Dinâmica" para descrever as velocidades que podem ser atingidas no próximo intervalo de tempo, dadas as limitações de torque/aceleração dos motores.
 - a. **No Artigo:** O foco é em robôs de alta velocidade onde a inércia importa.
 - b. **No Código:** O cálculo de **Vd** usa **config.max_accel * config.dt** para limitar a busca, garantindo que o robô não tente "mudar de ideia" mais rápido do que os seus motores permitem.

Pontos Levantados no arquivo de Janela Dinâmica:

O arquivo **JanelaDinamica.pdf** trabalha com o estabelecimento de que o DWA é uma estratégia de desvio de obstáculos reativa, o que significa que o robô toma decisões baseadas no seu estado atual e sensores, operando no espaço de velocidades (linear e angular). Um ponto central abordado é o modelo cinemático, onde o movimento do robô é descrito por arcos de circunferência em pequenos intervalos de tempo, garantindo que o plano respeite a física real do veículo.

O documento detalha como o algoritmo filtra as velocidades para criar a "Janela Dinâmica" propriamente dita. Esse processo envolve três camadas de restrição:

- Limites físicos do hardware;
- Velocidades que podem ser alcançadas no próximo ciclo de tempo considerando a aceleração atual;
- Segurança, mantendo apenas as velocidades admissíveis que permitem ao robô frear totalmente antes de colidir com qualquer obstáculo detectado.

Esta filtragem é essencial para que o robô não apenas planeje um caminho, mas garanta que consegue executá-lo com segurança.

Na parte prática, o PDF descreve a estrutura do código no toolbox *PythonRobotics*, destacando a classe de configuração onde são definidos parâmetros como o ganho dos custos e as dimensões do robô. O resumo explica que a escolha do movimento ideal é feita através de uma função de custo que **busca o equilíbrio entre três objetivos: a progressão em direção ao alvo (minimizando o erro angular), a manutenção de uma velocidade alta para garantir eficiência e a maximização da distância em relação aos obstáculos.**

Pontos teóricos que são abordados no código base do robô em implementação:

1. Definição do Espaço de Busca (A Janela Dinâmica): o relatório destaca que o DWA reduz as velocidades possíveis para aquelas que o robô consegue realmente alcançar. No código, isso é feito na função **calc_dynamic_window**:

```
109  def calc_dynamic_window(x, config):  
110      """  
111      calculation dynamic window based on current state x  
112      """
```

a. **Velocidades Físicas (Vs):** Definidas pelos limites de hardware:

```
114      # Dynamic window from robot specification  
115      Vs = [config.min_speed, config.max_speed,  
116            -config.max_yaw_rate, config.max_yaw_rate]
```

b. **Velocidades Alcançáveis (Vd):** Calculadas com base na aceleração e no intervalo de tempo (dt).

```

118         # Dynamic window from motion model
119         Vd = [x[3] - config.max_accel * config.dt,
120              x[3] + config.max_accel * config.dt,
121              x[4] - config.max_delta_yaw_rate * config.dt,
122              x[4] + config.max_delta_yaw_rate * config.dt]
123

```

- c. **Interseção (Janela Final):** O código seleciona o valor mais restritivo entre os dois.

```

124         # [v_min, v_max, yaw_rate_min, yaw_rate_max]
125         dw = [max(Vs[0], Vd[0]), min(Vs[1], Vd[1]),
126              max(Vs[2], Vd[2]), min(Vs[3], Vd[3])]
127

```

2. Modelo de Movimento (Cinemática)

```

95  def motion(x, u, dt):
96      """
97      motion model
98      """
99
100     x[2] += u[1] * dt
101     x[0] += u[0] * math.cos(x[2]) * dt
102     x[1] += u[0] * math.sin(x[2]) * dt
103     x[3] = u[0]
104     x[4] = u[1]
105
106     return x

```

3. Predição de Trajetória

```

139     while time <= config.predict_time:
140         x = motion(x, [v, y], config.dt)
141         trajectory = np.vstack((trajectory, x))
142         time += config.dt
143
144     return trajectory

```

4. Implementação da Função de Custo

```

161     trajectory = predict_trajectory(x_init, v, y, config)
162     # calc cost
163     to_goal_cost = config.to_goal_cost_gain * calc_to_goal_cost(trajectory, goal)
164     speed_cost = config.speed_cost_gain * (config.max_speed - trajectory[-1, 3])
165     ob_cost = config.obstacle_cost_gain * calc_obstacle_cost(trajectory, ob, config)
166
167     final_cost = to_goal_cost + speed_cost + ob_cost

```

5. Verificação de Segurança (Velocidades Admissíveis)

```

210     if np.array(r <= config.robot_radius).any():
211         return float("Inf")

```

Teoricamente a implementação está fiel à teoria. A implementação cobre desde a cinemática não-holonômica (o robô não se move lateralmente, ele precisa girar) até a dinâmica de aceleração.

Pontos levantados no arquivo PythonRobotics

1. Foco na Legibilidade e Educação. O objetivo principal do PythonRobotics é ser uma ferramenta para iniciantes entenderem a lógica por trás de algoritmos complexos. No código, isso é implementado através de:
 - a. Módulos Padrão: O uso exclusivo de bibliotecas amplamente conhecidas como **numpy** (para cálculos matriciais) e **matplotlib** (para visualização), facilitando a leitura sem a necessidade de bibliotecas proprietárias ou complexas.
 - b. Código "Flat": A lógica é escrita de forma direta, evitando abstrações excessivas, o que permite seguir o fluxo do algoritmo de cima para baixo.
2. Animações Intuitivas para Validação: O projeto defende que a melhor forma de entender a navegação é vendo o robô agir.
 - a. No código base, a variável **show_animation = True** e as funções **plot_robot** e **plot_arrow** servem exatamente para este propósito. Elas permitem que o utilizador veja não apenas o caminho final, mas todas as **trajetórias candidatas** (em verde) que o robô avaliou antes de tomar uma decisão.
3. Implementação de Algoritmos "Práticos e Amplamente Utilizados": O PythonRobotics seleciona apenas algoritmos que têm aplicação real na indústria e academia :
 - a. O DWA é um exemplo clássico disso. O ponto levantado pelo PDF é que o código deve ser uma **referência funcional**. A implementação fornecida não é apenas teórica; ela lida com problemas práticos como a prevenção de colisões em tempo real e a chegada precisa ao objetivo (**dist_to_goal <= config.robot_radius**).
4. Modularidade e Configuração (**Classe Config**): Um padrão do PythonRobotics observado no código é a separação dos parâmetros do algoritmo da lógica de execução:
 - a. A classe **Config** centraliza tudo o que define o comportamento do robô (velocidade, aceleração, ganhos de custo). Isso permite que o utilizador altere o comportamento do robô (tornando-o mais cauteloso ou mais rápido) sem precisar mexer nas funções matemáticas de controle.
5. Simulação de Diferentes Modelos de Robô: O projeto visa suportar diferentes morfologias. No código, isso é visível na implementação da **RobotType**:
 - a. O script permite alternar entre um robô **Circular** e um **Retangular**. Isso altera a lógica de verificação de colisão (**calc_obstacle_cost**), mostrando como o algoritmo DWA se

adapta ao formato físico do robô, um ponto avançado mencionado na documentação do projeto.

Enquanto a lógica implementada no JanelaDinamica é a matemática, com todas as suas operações, no arquivo PythonRobotics o foco é a arquitetura de software, transformando a base matemática em algo acessível, visual e testável, pontos que estão perfeitamente representados .

Conclusão final:

Embora a implementação baseada em Fox et al. (1997) e no PythonRobotics seja matematicamente sólida, em um cenário de uso real ou simulações mais complexas, o código pode não funcionar "perfeitamente", podendo ser causas desse não funcionamento ideal:

1. O Problema dos "Mínimos Locais" (Armadilhas em forma de U) :

O DWA é um algoritmo **reativo**, o que significa que ele olha apenas para o futuro imediato (o **predict_time** de 3 segundos).

- **O Motivo:** Se o robô entrar em um obstáculo em forma de "U" ou um corredor sem saída que aponte para o objetivo, ele pode ficar oscilando ou preso. Ele quer ir para o alvo (que está à frente), mas o custo de obstáculo o impede, e ele não tem uma visão global para entender que precisa dar a volta.
- No Código: A função **calc_to_goal_cost** foca apenas no erro angular para o alvo final, sem considerar um planejamento de rotas de longo alcance.

2. Escolha dos Pesos (Ganhos de Custo) :

O Motivo: Se o **obstacle_cost_gain** for muito baixo, o robô passa perigosamente perto de objetos. Se o **speed_cost_gain** for muito alto, ele pode ignorar curvas suaves para manter a velocidade máxima, resultando em colisões ou trajetórias instáveis. Encontrar o equilíbrio exige tentativa e erro (tuning).

3. Resolução da Amostragem (**v_resolution** e **yaw_rate_resolution**):

O Motivo: Se a resolução for muito baixa (valores grandes), o robô pode não encontrar a velocidade "perfeita" para desviar de um obstáculo apertado e acreditar que está bloqueado. Se for muito alta, o computador pode travar ou o robô demorar a responder, pois o número de trajetórias para calcular cresce exponencialmente.

4. Modelo de "Curvatura Constante" em Ambientes Dinâmicos: Fox assume que a velocidade (v, ω) se mantém constante durante todo o tempo de predição.

O Motivo: No mundo real, obstáculos podem se mover. O código assume que os obstáculos estão parados (ob é um array estático). Se um obstáculo se mover em direção ao robô, a predição de 3 segundos estará errada, e a colisão será inevitável porque o robô não "previu" o movimento do objeto.

5. Geometria do Robô vs. Obstáculos Pontuais:

O Motivo: Se um obstáculo for uma parede longa e fina, mas representada apenas por dois pontos distantes no array ob, o robô pode tentar "atravessar" o meio da

parede, pois matematicamente a distância para os *pontos* (as extremidades) ainda é grande o suficiente.

6. Latência de Controle : O DWA assume que o comando enviado será executado instantaneamente.

O Motivo: Se o dt (tempo de ciclo) for muito grande ou se houver atraso no processamento, o robô estará em uma posição diferente daquela que o algoritmo calculou quando o comando finalmente chegar aos motores. Isso causa oscilações e instabilidade.